CODING SHUTTLE

Docker

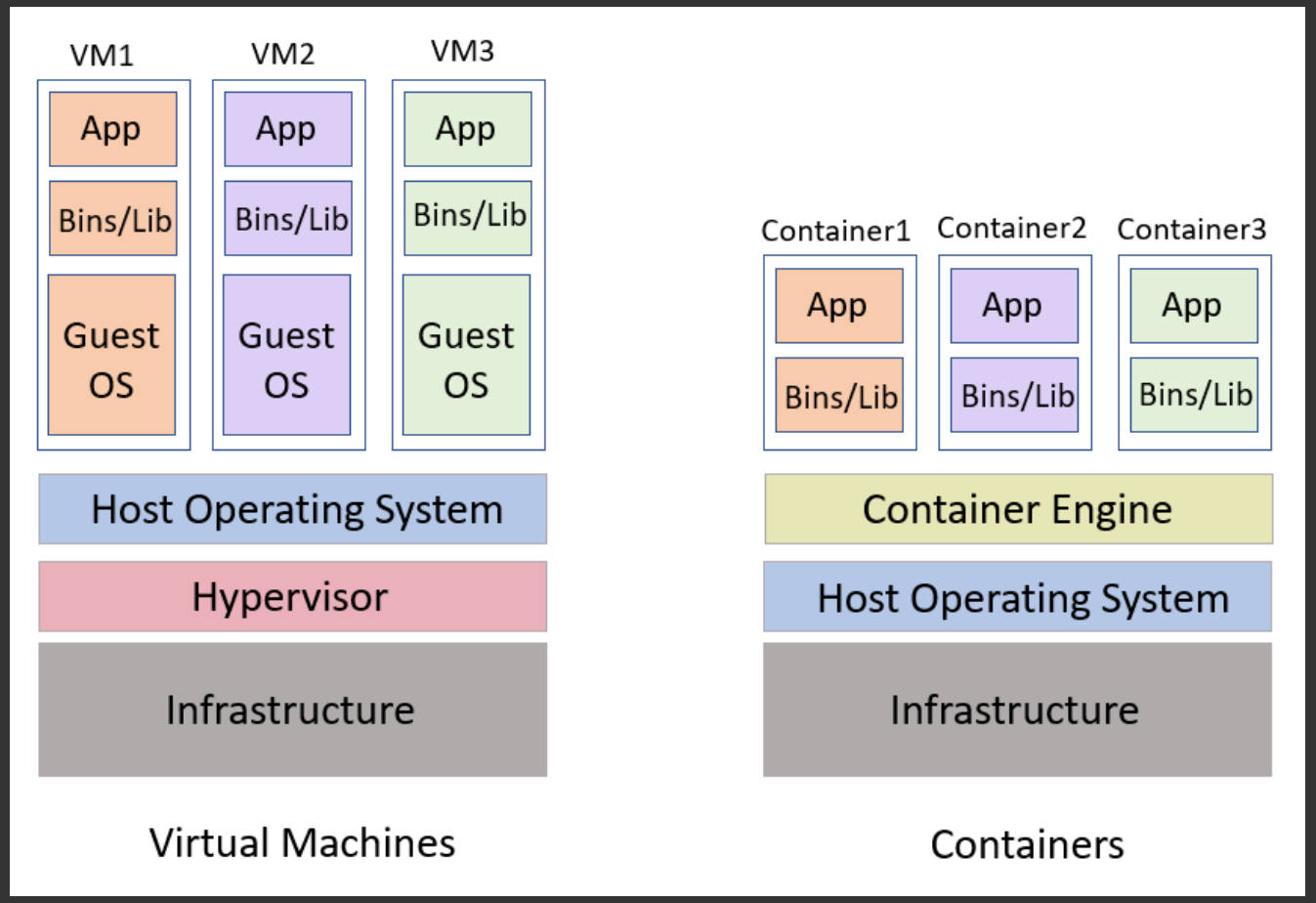
Introduction to Docker

The Problem Statement

How do you allow multiple applications to run on one machine at the same time?
You can use Virtual Machines, but Virtual Machines are resource-heavy. Which can lead to the following problems:

- Efficient Resource Utilization: Virtual Machines are heavy
- Application Isolation and Security: VM hosts file-system crashing into each other
- Scalability: Virtual machines inefficient resource management

Enters Containers



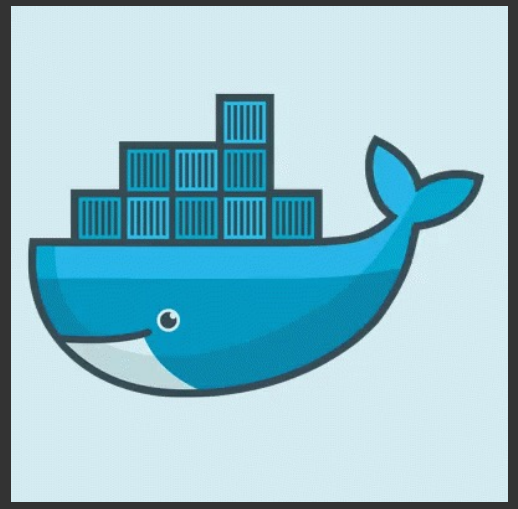
Containers share the host operating system's (OS) kernel and are lightweight, with minimal overhead. They provide fast startup times and efficient resource utilization.

But What is Docker?

Docker is the platform that enables the creation, management, and execution of containers. Containers handle the isolation, packaging, and consistent environment, but Docker provides the tools and infrastructure that make working with containers easy and efficient.

Docker as a Platform provides:

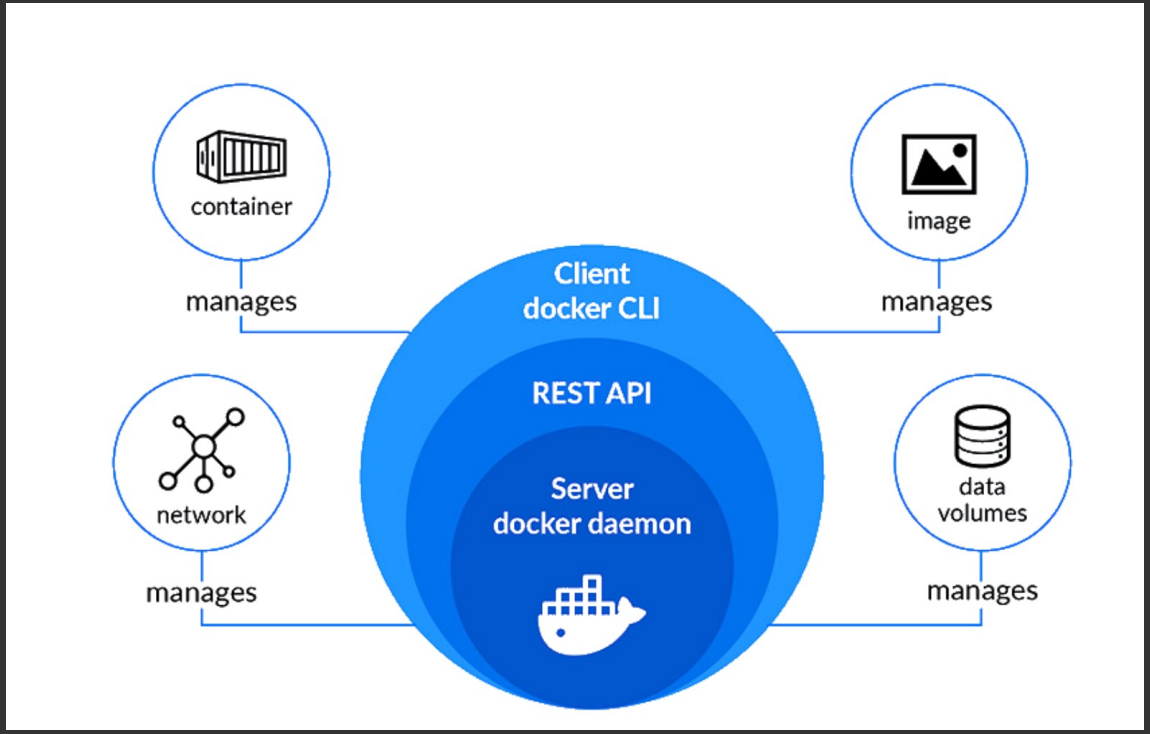
- Docker Runtime
- Image creation and management
- Networking
- Security
- Volume and Storage Management



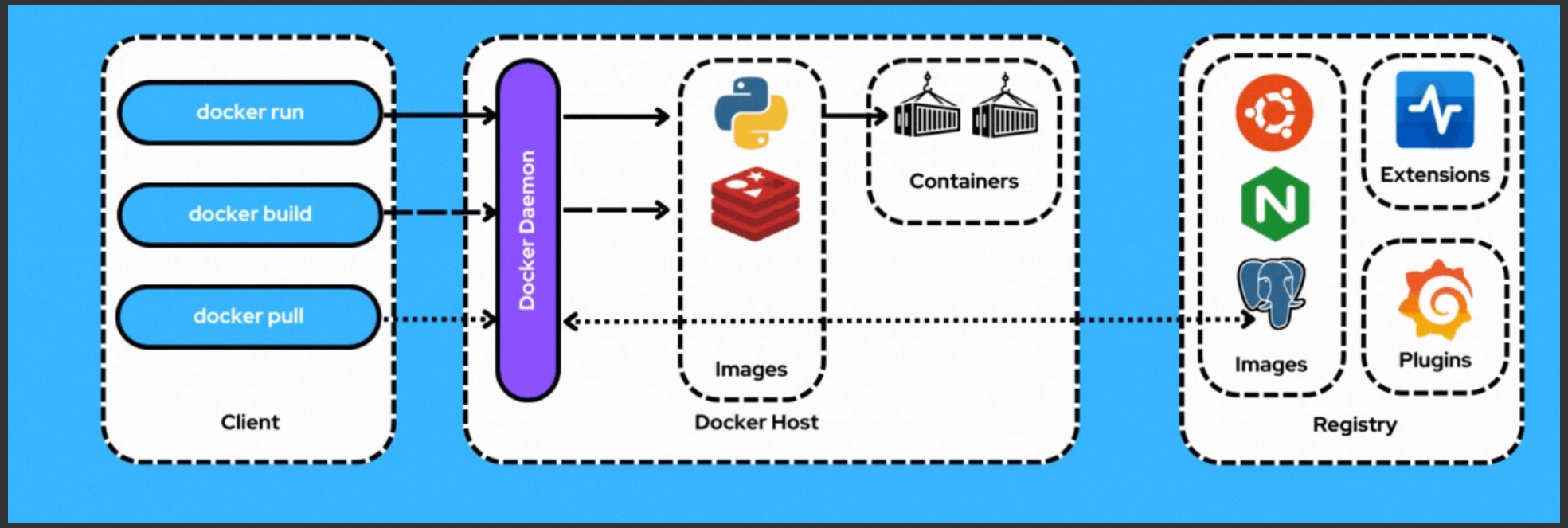
How Docker Works

The Docker client talks to the Docker daemon, which does the heavy lifting of building and running your Docker containers. The Docker daemon (*dockerd*) listens for Docker API requests and manages Docker objects such as images, containers, networks, and volumes.

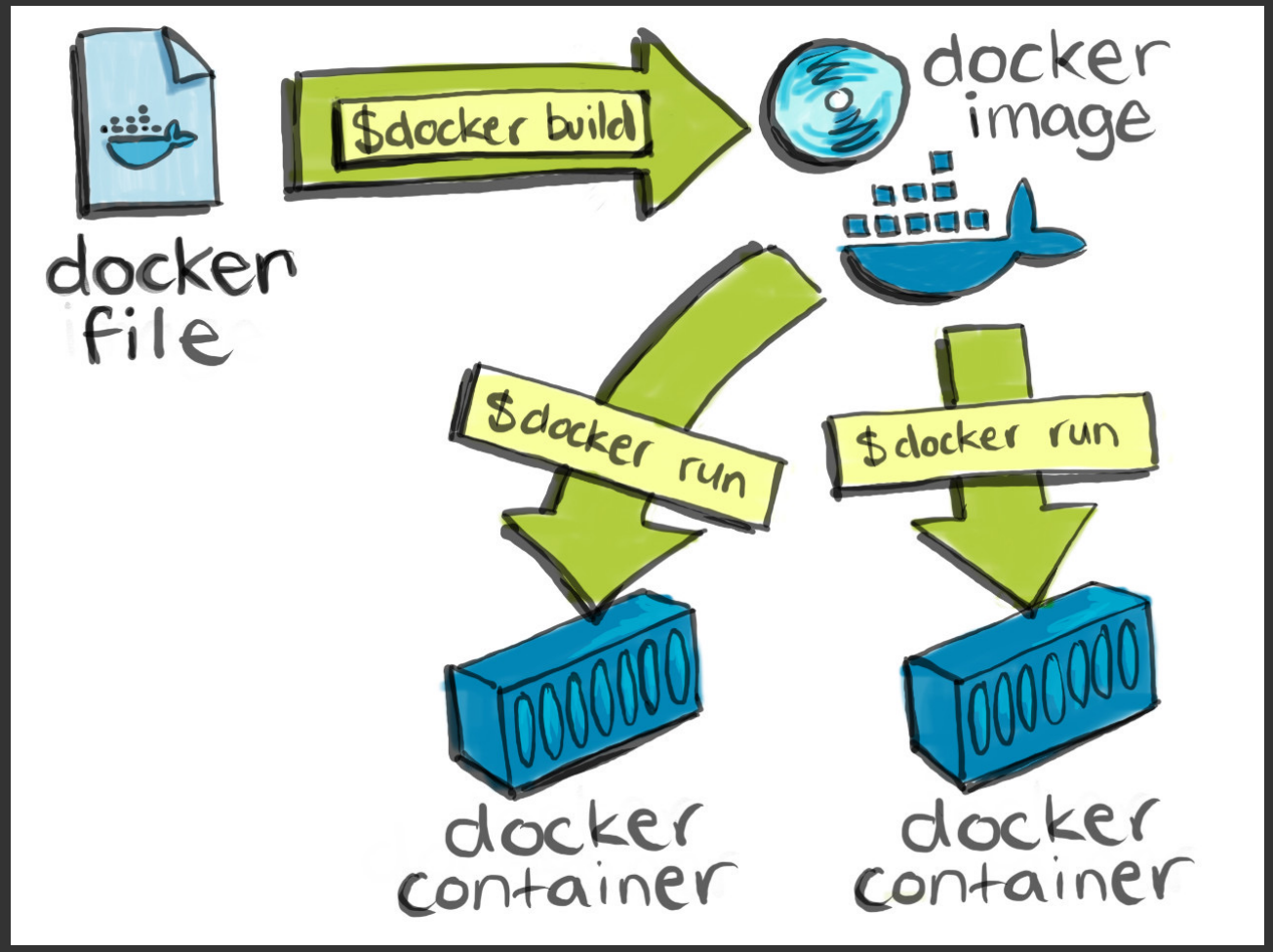
The Docker client and daemon can run on the same or different system. The Docker client and daemon communicate using a REST API, over UNIX sockets or a network interface.



Docker Architecture



Dockerfile



A Dockerfile serves as the recipe for building a Docker image. It contains instructions that specify how to construct the image, including the base image to use, environment variables, dependencies to install, and commands to execute.

Docker Image

Once a Dockerfile is created, the next step is to build the Docker image using the docker build command. This command reads the instructions in the Dockerfile and executes them sequentially, layer by layer, to construct the image. Each instruction in the Dockerfile creates a new image layer, which is cached for subsequent builds to improve build performance.

During the build process, Docker pulls the necessary base image layers from a registry (e.g., Docker Hub) and executes the instructions in the Dockerfile to customize the image. Once the build is complete, Docker generates a new image with a unique identifier known as a digest.

Docker Image vs Containers

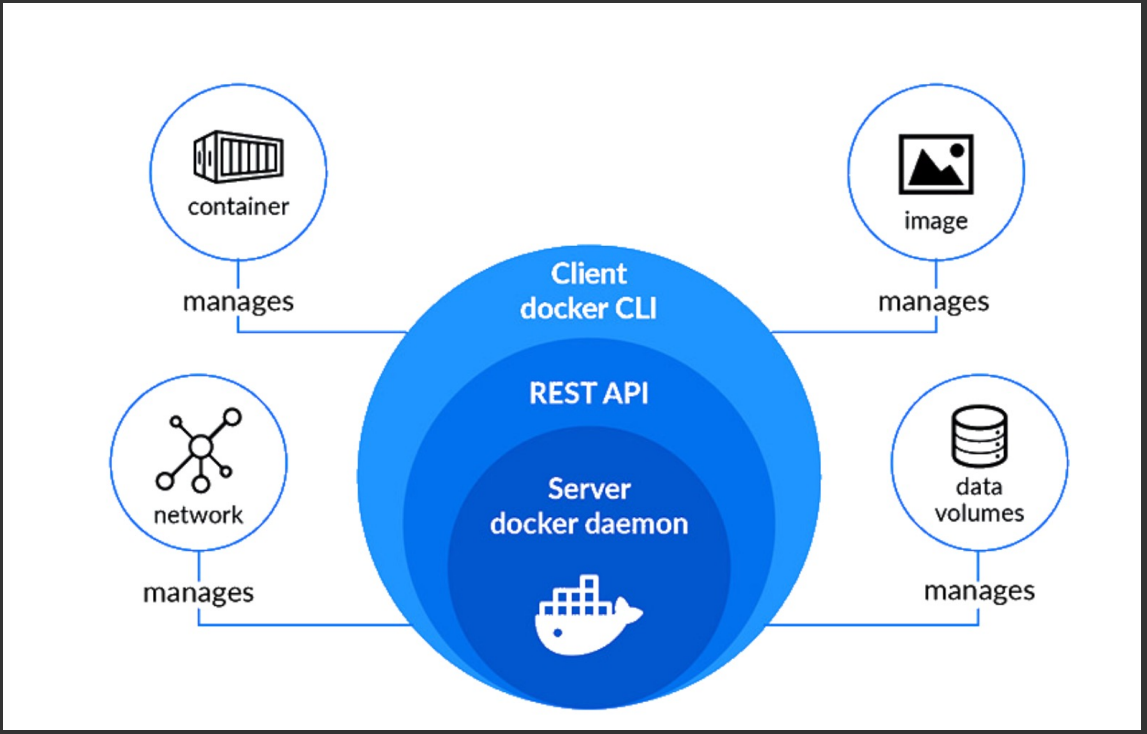
While Docker images serve as the blueprint for containers, containers are the runtime instances of those images. Containers are isolated environments that run the application specified in the image.

In essence, images are static, immutable artifacts, while containers are dynamic, running instances that can be started, stopped, and destroyed.

Benefits of Docker

- Efficient Resource Utilization
- Application Isolation & Security
- Scalability
- Portability
- Consistency

Docker allows you to ship, test, and deploy your applications in any environment without worrying about incompatibility issues, regardless of the machine's configuration settings.



The Problem Statement

How do you allow multiple applications to run on one machine at the same time?
You can use Virtual Machines, but Virtual Machines are resource-heavy. Which can lead to the following problems:

- Efficient Resource Utilization: Virtual Machines are heavy
- Application Isolation and Security: VM hosts file-system crashing into each other
- Scalability: Virtual machines inefficient resource management